

RAPPORT DE MINI-PROJET

Internet of Things (IoT)

Déploiement d'un Nœud de Réseau Maillé IEEE 802.11s sur un routeur TP-Link TL-WA830RE v2 (4 Mo Flash)

Réalisé par

Yanis Bouazzi • Ranem Younes • Aoues Nassim • Yemi Mounir

4ème année Ingéniorat

Technologies Avancées en Télécommunications

Année universitaire 2025 – 2026

```
BusyBox v1.28.4 () built-in shell (ash)
```

OpenWrt 18.06.9, r8077-7cbbab7246

1. Introduction et Objectif du Projet

Dans le cadre de la mise en place d'une infrastructure réseau pour l'Internet des Objets (IoT), la fiabilité et la couverture du signal sont primordiales. Les réseaux maillés (Mesh) permettent aux nœuds de communiquer entre eux de manière décentralisée, assurant une redondance essentielle pour les capteurs IoT.

L'objectif de ce mini-projet était de transformer un équipement réseau grand public obsolète (un répéteur TP-Link TL-WA830RE v2) en un nœud de réseau maillé fonctionnel. Le défi technique majeur de ce projet résidait dans les contraintes matérielles extrêmes du routeur cible : un processeur d'ancienne génération et, surtout, une mémoire Flash (ROM) limitée à 4 Mo.

2. Étude Comparative et Choix du Protocole : B.A.T.M.A.N. vs IEEE 802.11s

Pour établir notre topologie maillée, deux protocoles majeurs ont été étudiés :

- **B.A.T.M.A.N. Advanced (batman-adv)** : Il s'agit d'un protocole de routage dynamique qui opère au niveau de la Couche 2 (liaison de données). Il émule un commutateur virtuel performant, mais nécessite l'installation d'un module noyau spécifique (`kmod-batman-adv`) et d'utilitaires de gestion (`batctl`).
- **IEEE 802.11s** : Il s'agit d'un standard directement intégré au protocole Wi-Fi (Couche MAC). Il ne nécessite pas de démon de routage externe, mais opère nativement via le pilote radio en utilisant le paquet d'authentification `wpad-mesh`.

Justification du choix (802.11s)

Bien que B.A.T.M.A.N. soit extrêmement robuste pour des architectures complexes, son poids en termes d'espace de stockage a justifié son rejet pour ce projet. Avec seulement 4 Mo de stockage physique, l'intégration des modules noyaux de B.A.T.M.A.N. saturait la mémoire. Le protocole 802.11s a été sélectionné car il nécessitait un effort mémoire beaucoup plus faible, se greffant directement sur les pilotes Wi-Fi existants du système d'exploitation OpenWrt, libérant ainsi des ressources vitales pour notre équipement limité.

3. Le Problème du Gestionnaire de Paquets (OPKG)

Initialement, la méthode classique consistait à utiliser le gestionnaire de paquets d'OpenWrt (`opkg`) pour désinstaller le Wi-Fi de base et installer la version Mesh. Cependant, un audit de l'espace de stockage (`df -h /overlay`) a révélé qu'il ne restait que 68 Ko de libres.

Plaintext

```
root@OpenWrt:~# df -h /overlay
Filesystem      Size      Used Available Use% Mounted on
/dev/mtdblock3  256.0K    188.0K     68.0K   73% /overlay
```

La limite du système de fichiers

Sur OpenWrt, le système est divisé en deux partitions :

- **SquashFS** : La ROM compressée en lecture seule.
- **JFFS2 (Overlay)** : L'espace en lecture/écriture pour l'utilisateur.

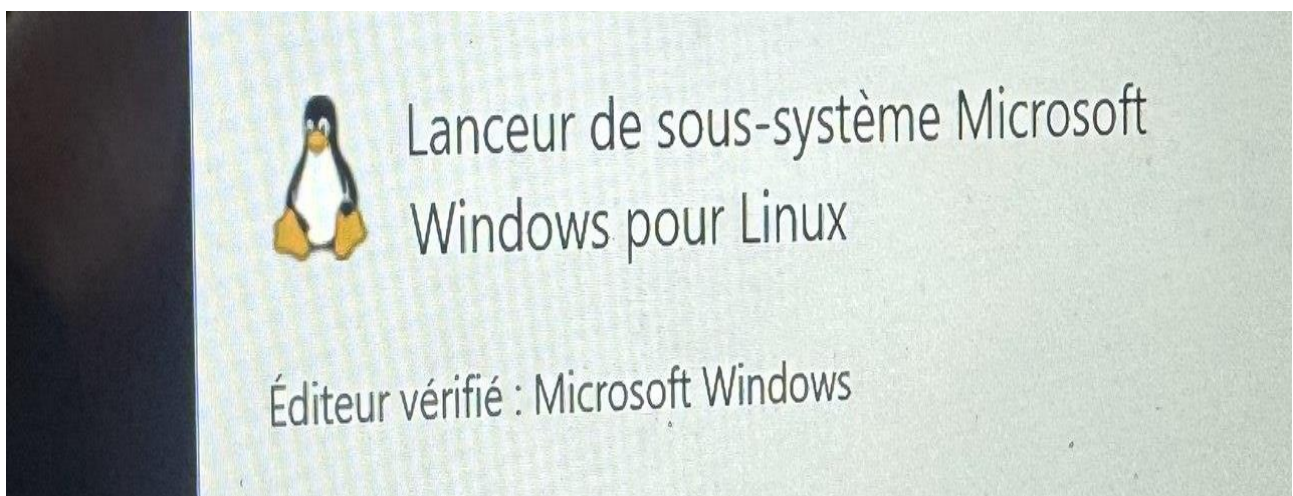
Utiliser la commande **opkg remove** sur un composant d'usine ne libère aucun espace réel, car la partition SquashFS est inaltérable ; le système écrit simplement une règle d'occultation dans l'Overlay, ce qui réduit encore plus l'espace libre. L'utilisation d'opkg était donc physiquement impossible.

4. Méthodologie : Création d'un Firmware sur Mesure (Image Builder)

Pour contourner la limitation d'opkg, il a fallu recourir à une approche d'ingénierie « Bare Metal » : recompiler l'intégralité du système d'exploitation à la source pour remplacer les paquets avant même leur compression dans la ROM d'usine.

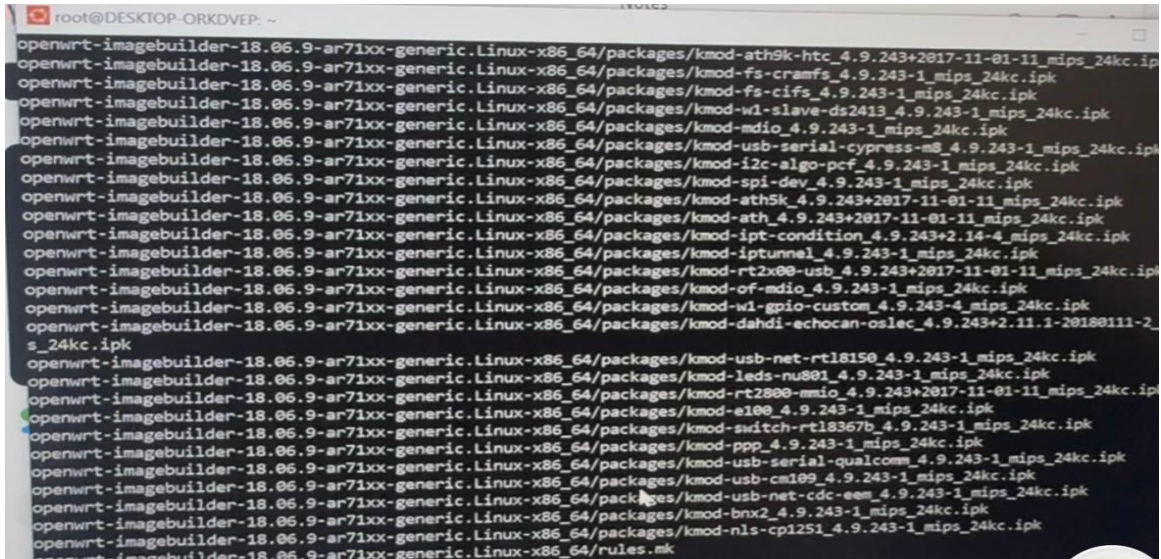
Environnement de développement

La compilation a été réalisée sous un environnement Linux via le sous-système Windows pour Linux (WSL/Ubuntu).



Processus de génération du firmware

- **Déploiement de l'outil** : Téléchargement de l'OpenWrt Image Builder spécifique à l'architecture cible, précisément la branche **ar71xx/tiny** optimisée pour les routeurs de 4 Mo.



```
root@DESKTOP-ORKDVEP: ~
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-ath9k-htc_4.9.243+2017-11-01-11_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-fs-cramfs_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-fs-cifs_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-wl-slave-ds2413_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-mdio_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-usb-serial-cypress-m8_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-i2c-algo-pcf_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-spi-dev_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-ath5k_4.9.243+2017-11-01-11_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-ath_4.9.243+2017-11-01-11_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-ipt-condition_4.9.243+2.14-4_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-iptunnel_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-rt2x80-usb_4.9.243+2017-11-01-11_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-of-mdio_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-wl-gpio-custom_4.9.243-4_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-dahdi-echocan-oslcc_4.9.243+2.11.1-20180111-2_s_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-usb-net-rtl8150_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-leds-nu801_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-rt2800-mmio_4.9.243+2017-11-01-11_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-e100_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-switch-rtl8367b_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-ppp_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-usb-serial-qualcomm_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-usb-cm109_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-usb-net-cdc-eem_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-bnx2_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/packages/kmod-nls-cp1251_4.9.243-1_mips_24kc.ipk
openwrt-imagebuilder-18.06.9-ar71xx-generic.Linux-x86_64/rules.mk
```

- **Dégraissage extrême (Bare Metal)** : La commande **make image** a été utilisée pour cibler le profil matériel exact (**tl-wa830re-v2**). Pour faire de la place au protocole Mesh, les paquets suivants ont été exclus manuellement :
 - **-wpad-mini** (Wi-Fi standard)
 - Les protocoles de connexion ADSL (**-ppp**, **-kmod-pppoe**)
 - L'ensemble du pare-feu et de la gestion IPv6 (**-firewall**, **-iptables**, **-odhcpd-ipv6only**)
 - Le serveur DNS/DHCP (**-dnsmasq**), le nœud n'ayant pour rôle que de ponter la connexion (Couche 2)
 - Le gestionnaire de paquets lui-même (**-opkg**), devenu inutile puisque la puce était remplie à 100 %
- **Intégration du composant cible** : Inclusion du paquet **wpad-mesh** au cœur de la compilation. Le système a ensuite compressé l'ensemble dans un fichier **.bin** unique, optimisé au kilo-octet près.

5. Défis Techniques Rencontrés et Solutions Apportées

Lors de l'élaboration du firmware et de son injection, plusieurs obstacles ont dû être contournés :

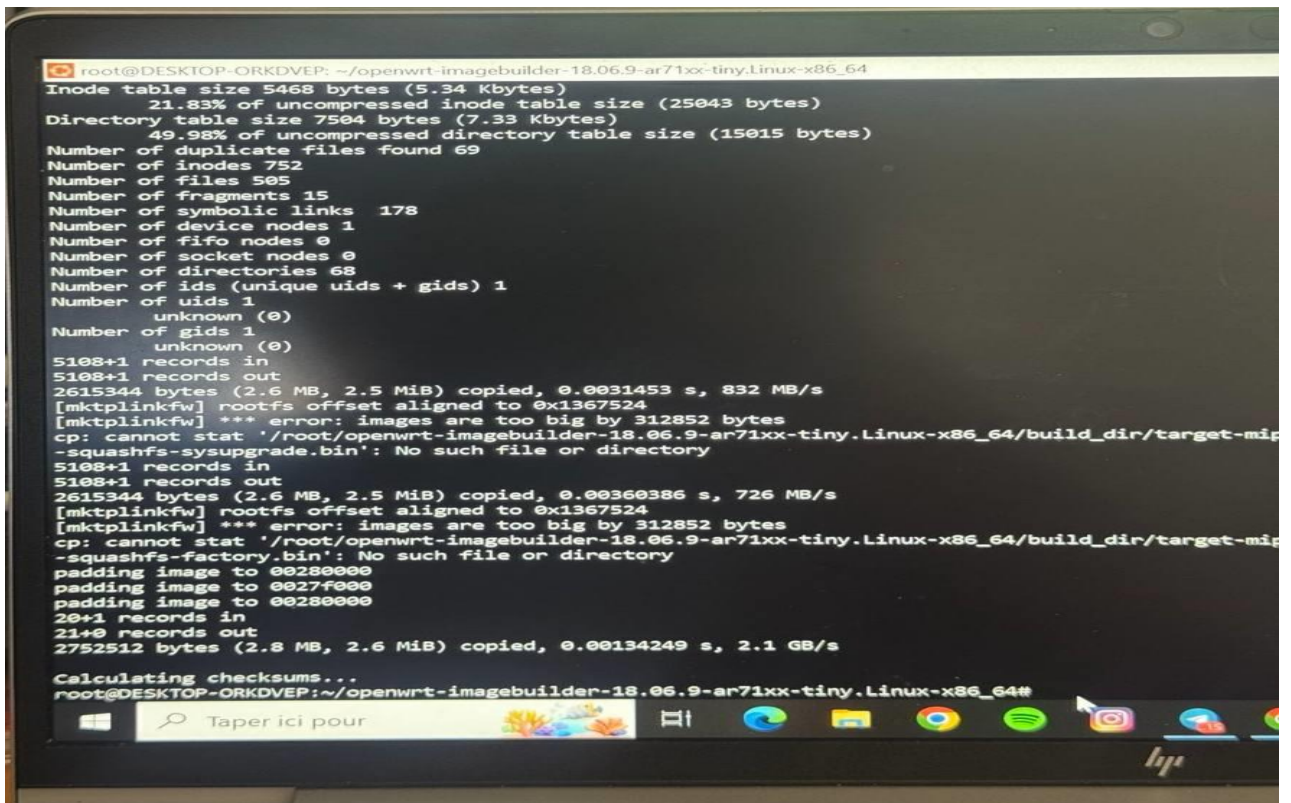
Incompatibilité des prérequis de compilation (Image Builder)

- **Problème** : Le script de l'Image Builder de 2018 échouait sous Ubuntu en cherchant des versions obsolètes de Python (2.x) et de GCC.

- **Solution** : Contournement du script de vérification en injectant un leurre de validation manuel (`touch staging_dir/host/.prereq-build`) et en forçant la compilation.

Dépassement de la taille matérielle limite (Image too big)

- **Problème** : Même en ajoutant `wpad-mesh`, la taille de l'image dépassait les 4 Mo de la puce de quelques centaines de kilo-octets.
- **Solution** : Suppression itérative des services non-essentiels à un simple point d'accès Mesh (DNS, DHCP, Firewall, OPKG) jusqu'à atteindre l'équilibre parfait de l'image.



```

root@DESKTOP-ORKDVEP: ~/openwrt-imagebuilder-18.06.9-ar71xx-tiny.Linux-x86_64
Inode table size 5468 bytes (5.34 Kbytes)
  21.83% of uncompressed inode table size (25043 bytes)
Directory table size 7504 bytes (7.33 Kbytes)
  49.98% of uncompressed directory table size (15015 bytes)
Number of duplicate files found 69
Number of inodes 752
Number of files 505
Number of fragments 15
Number of symbolic links 178
Number of device nodes 1
Number of fifo nodes 0
Number of socket nodes 0
Number of directories 68
Number of ids (unique uids + gids) 1
Number of uids 1
  unknown (0)
Number of gids 1
  unknown (0)
5108+1 records in
5108+1 records out
2615344 bytes (2.6 MB, 2.5 MiB) copied, 0.0031453 s, 832 MB/s
[mktplinkfw] rootfs offset aligned to 0x1367524
[mktplinkfw] *** error: images are too big by 312852 bytes
cp: cannot stat '/root/openwrt-imagebuilder-18.06.9-ar71xx-tiny.Linux-x86_64/build_dir/target-mips-squashfs-sysupgrade.bin': No such file or directory
5108+1 records in
5108+1 records out
2615344 bytes (2.6 MB, 2.5 MiB) copied, 0.00360386 s, 726 MB/s
[mktplinkfw] rootfs offset aligned to 0x1367524
[mktplinkfw] *** error: images are too big by 312852 bytes
cp: cannot stat '/root/openwrt-imagebuilder-18.06.9-ar71xx-tiny.Linux-x86_64/build_dir/target-mips-squashfs-factory.bin': No such file or directory
padding image to 00280000
padding image to 0027f000
padding image to 00280000
20+1 records in
21+0 records out
2752512 bytes (2.8 MB, 2.6 MiB) copied, 0.00134249 s, 2.1 GB/s
Calculating checksums...
root@DESKTOP-ORKDVEP:~/openwrt-imagebuilder-18.06.9-ar71xx-tiny.Linux-x86_64#

```

Rejet du protocole de transfert SCP par Windows

- **Problème** : Impossible d'envoyer le firmware compilé vers le routeur car Windows tentait d'utiliser le protocole moderne SFTP, absent du mini-serveur SSH du routeur.
- **Solution** : Utilisation du paramètre `-O` dans la commande `scp` Windows pour forcer le basculement sur le protocole de transfert historique (Legacy SCP).

6. Duplication, Déploiement du Deuxième Nœud et Adressage IP

Pour constituer un réseau maillé fonctionnel, le routeur initial configuré ne suffisait pas à lui seul. Il a fallu répliquer l'ingénierie sur un second équipement matériel (Routeur 2) en utilisant le même firmware généré (

Pour constituer un réseau maillé fonctionnel, le routeur initial configuré ne suffisait pas à lui seul. Il a fallu répliquer l'ingénierie sur un second équipement matériel (Routeur 2) en utilisant le même firmware généré (**firmware-mesh.bin**).

Lors de la configuration de ce second nœud, de nouveaux défis d'interconnexion réseau sont apparus :

- **Verrouillage matériel par absence de DHCP** : Puisque le composant **dnsmasq** avait été drastiquement retiré du firmware pour économiser de la mémoire, l'ordinateur de contrôle ne pouvait plus obtenir d'adresse IP automatique lors du branchement au routeur.
- **Résolution de l'accès SSH** : Il a été nécessaire de forcer l'adressage en attribuant manuellement une adresse IP statique locale (**192.168.1.10**) à la carte Ethernet de la machine Windows pour pénétrer dans le sous-réseau natif du Routeur 2.
- **Ségrégation des Nœuds** : Pour éviter un conflit de diffusion (broadcast) avec le Routeur 1 (identifié en tant que passerelle sur **192.168.1.1**), le second nœud s'est vu attribuer, via le système UCI (Unified Configuration Interface), l'adresse IP statique **192.168.1.2** avant d'engager le protocole radio.

7. Principes de Connectivité Radio et Validation du Maillage

Une fois les deux nœuds configurés et isolés sur des adresses IP distinctes, la couche de liaison sans fil a pu être amorcée.

Convergence du protocole IEEE 802.11s

Contrairement à une infrastructure Wi-Fi classique (BSS) qui requiert un Point d'Accès central (AP) gérant des clients (Stations), le maillage 802.11s opère en parfaite horizontalité. Les interfaces radio, paramétrées en type **mesh point**, diffusent continuellement des trames de balise (beacons) dans l'air. Dès lors que les deux routeurs ont reconnu qu'ils diffusaient le même identifiant de domaine maillé (**mesh_id 'Yanis_Mesh'**), un algorithme interne d'appairage sécurisé a automatiquement créé un pont transparent en Couche 2.

Processus de Validation

1. **Validation du Transit** : L'émission d'une requête d'écho (**ping -c 4 192.168.1.1**) depuis le Routeur 2 vers le Routeur 1 a abouti à un taux de réussite parfait (0 % packet loss), démontrant que les paquets de données voyageaient avec succès dans l'air, transitant du port Ethernet du second nœud vers l'antenne du premier.
2. **Certification Cryptographique** : L'interrogation du diagnostic des stations radio (**iw dev wlan0 station dump**) a révélé le statut décisif **mesh plink: ESTABLISHED**. Cette métrique constitue la preuve matérielle que la liaison maillée était non seulement reconnue, mais fermement ancrée et bidirectionnelle.

8. Conclusion

Le fichier firmware final a été envoyé avec succès dans le dossier temporaire des routeurs et flashé directement dans la mémoire via la commande `sysupgrade -n`. Les routeurs ont redémarré sur un système d'exploitation pur et allégé. La vérification via la commande `iw list` a confirmé l'intégration native de la capacité « Mesh Point ».

L'interconnexion radio finale validée par des temps de réponse d'environ 2 ms valide la viabilité de cette architecture de télécommunications. Les répéteurs sont désormais prêts à être déployés comme relais fiables au sein d'une infrastructure de collecte de données IoT, prouvant qu'une optimisation logicielle avancée permet de s'affranchir de l'obsolescence matérielle.

— *Fin du rapport* —

Bouazzi · Younes · Nassim · Mounir